



**Engenharia de Computação
Inteligência Artificial
04 - Estruturas e Estratégias de Busca
Parte II - Busca Local e Ambientes
Complexos**

Organização da aula

- Retomada: busca clássica, função heurística e A^* .
- Busca local e otimização: hill climbing, simulated annealing, busca em feixe e algoritmos genéticos.
- Busca com ações não determinísticas e planos de contingência.
- Busca com observação parcial e estados de crença.
- Busca online em ambientes inicialmente desconhecidos.

Fio condutor

Como a estratégia de busca deve mudar quando o caminho deixa de ser o principal objetivo ou quando o agente não conhece perfeitamente o efeito das ações e o estado do ambiente?

Quando o estado final é mais importante que o caminho

- Em problemas de otimização, muitas vezes interessa a qualidade do estado final, não a sequência exata de estados percorridos.
- Exemplos: disposição de componentes, escalonamento, configuração de parâmetros e problemas combinatórios.
- A busca local mantém um ou poucos estados candidatos e se desloca pela vizinhança.
- Isso reduz drasticamente o uso de memória.

Observação

Na busca local, a noção de “melhor” é definida por uma função objetivo ou função de avaliação do estado.

Busca local

- Começa em um estado inicial, frequentemente escolhido de forma aleatória.
- Gera estados vizinhos por pequenas modificações da solução atual.
- Escolhe como se mover usando a função objetivo.
- Pode operar em espaços discretos ou contínuos.
- Em geral, não mantém uma árvore completa de busca.

Pergunta

Se mantemos apenas o estado atual, o que acontece se a busca entrar em uma região ruim do espaço?

Hill Climbing — subida de encosta

- Avalia os vizinhos do estado atual.
- Move para um vizinho com valor melhor segundo a função objetivo.
- Repete até não existir melhoria local.
- É uma estratégia local e gulosa: usa apenas a informação disponível na vizinhança imediata.

Pergunta

Parar porque nenhum vizinho é melhor significa ter encontrado o melhor estado de todo o espaço?

Algoritmo Hill Climbing

function HILL_CLIMBING (*problem*) **returns** a state that is local maximum

current $\leftarrow$ MAKE-NODE (*problem*.INITIAL-STATE)

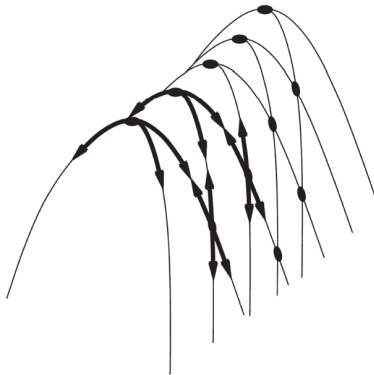
loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

if *neighbor*.VALUE < *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

Limitações do Hill Climbing



- **Ótimos locais:** estados melhores que todos os vizinhos, mas piores que o ótimo global.
- **Platôs:** regiões com pouca ou nenhuma variação da função objetivo.
- **Cordilheiras:** melhorias exigem sequências de movimentos que a vizinhança simples não captura bem.

Como melhorar o Hill Climbing?

- **Passos laterais:** permitir temporariamente movimentos de mesmo valor.
- **Escolha estocástica:** escolher probabilisticamente entre movimentos de melhoria.
- **Primeira escolha:** aceitar o primeiro vizinho que melhora o estado atual, útil quando há muitos vizinhos.
- **Reinício aleatório:** executar o algoritmo várias vezes a partir de estados iniciais diferentes.

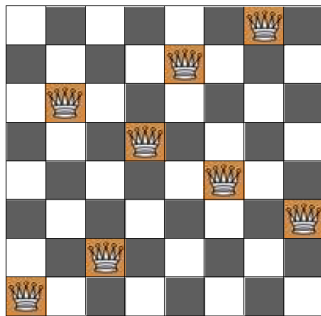
Discussão

Reiniciar a busca aumenta o custo computacional, mas reduz a dependência de uma única escolha inicial.

Exemplo: problema das 8 rainhas

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14		13	16	13	16
	14	17	15		14	16	16
17		16	18	15		15	
18	14		15	15	14		16
14	14	13	17	12	14	12	18

$h = 17$



$h = 1$ (mínimo local)

h = número de pares de rainhas que se atacam.

8 rainhas como problema de busca local

- Um estado pode ser representado pela posição de uma rainha em cada coluna.
- Um vizinho é obtido movendo uma rainha dentro da sua coluna.
- A função objetivo pode minimizar o número de pares de rainhas em conflito.
- O espaço possui $8^8 \approx 17$ milhões de configurações nessa formulação.
- Reinícios e passos laterais ajudam a superar ótimos locais e platôs.

Pergunta

Por que armazenar o caminho percorrido não é necessário para resolver esse problema?

Simulated Annealing

- Também explora a vizinhança, mas pode aceitar movimentos que pioram temporariamente a solução.
- A probabilidade de aceitar uma piora depende da magnitude da piora e de uma **temperatura T** .
- Em temperaturas altas, a busca explora mais livremente.
- À medida que T diminui, o comportamento se torna mais seletivo e próximo de hill climbing.

Observação

O objetivo é escapar de ótimos locais no início e estabilizar a busca progressivamente.

Algoritmo Simulated Annealing

function SIMULATED ANNEALING (*problem, schedule*) **returns** a solution state

inputs: *problem*, a problem

schedule, a mapping from time to “temperature”

```

current ← MAKE NODE (problem.INITIAL-STATE)

```

for $t \leftarrow 1$ **to** ∞ **do**
$$T \leftarrow \text{schedule}[t]$$

if $T = 0$ then return *current*

$next \leftarrow$ a randomly selected successor of $current$

$$\Delta E \leftarrow next.VALUE - current.VALUE$$

if $\Delta E > 0$ then *current* $\leftarrow$ *next*

else *current* $\leftarrow$ *next* only with probability $\exp(\Delta E/T)$

Busca em feixe local — Local Beam Search

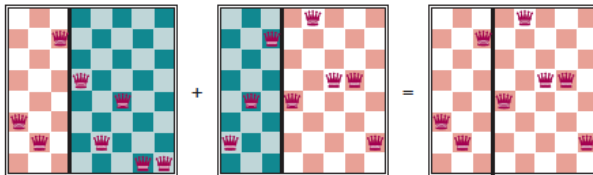
- Mantém simultaneamente k estados candidatos.
- Em cada iteração, gera sucessores dos k estados.
- Se alguma meta é encontrada, a busca termina.
- Caso contrário, conserva os k melhores sucessores.
- Os candidatos compartilham informação implicitamente porque competem pela mesma população de k estados.

Pergunta

Qual é o risco de os k estados se tornarem muito parecidos entre si?

Algoritmos genéticos: ideia geral

- Mantêm uma **população** de estados candidatos.
- Cada indivíduo possui uma representação da solução e um valor de aptidão.
- A população evolui por seleção, recombinação e mutação.
- A ideia central é combinar diversidade populacional e pressão por soluções de melhor qualidade.



Algoritmos genéticos: ciclo evolutivo

```

function GENETIC-ALGORITHM(population, fitness) returns an individual
  repeat
    weights  $\leftarrow$  WEIGHTED-BY(population, fitness)
    population2  $\leftarrow$  empty list
    for i = 1 to SIZE(population) do
      parent1, parent2  $\leftarrow$  WEIGHTED-RANDOM-CHOICES(population, weights, 2)
      child  $\leftarrow$  REPRODUCE(parent1, parent2)
      if (small random probability) then child  $\leftarrow$  MUTATE(child)
      add child to population2
    population  $\leftarrow$  population2
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to fitness

function REPRODUCE(parent1, parent2) returns an individual
  n  $\leftarrow$  LENGTH(parent1)
  c  $\leftarrow$  random number from 1 to n
  return APPEND(SUBSTRING(parent1, 1, c), SUBSTRING(parent2, c + 1, n))

```

Observação

Aqui, algoritmos genéticos aparecem como uma estratégia de busca/otimização populacional; seus operadores serão aprofundados quando necessário em outros contextos da disciplina.

Busca clássica ou busca local?

	Busca clássica	Busca local
Resultado	caminho até meta	estado de boa qualidade
Estrutura	árvore/gráfo	vizinhança
Memória	pode ser alta	normalmente baixa
Uso típico	planejamento/rotas	otimização
Risco	explosão combinatória	ótimo local

Pergunta

O problema das 8 rainhas pode ser resolvido pelas duas abordagens? O que muda na formulação e no objetivo da busca?

Atividade rápida: reconhecendo o tipo de busca

Atividade

Classifique cada problema como mais naturalmente associado a busca por caminho ou busca local e justifique:

- 1 rota entre duas cidades;
- 2 escalonamento de tarefas em máquinas;
- 3 configuração de uma rede com menor custo;
- 4 sequência de movimentos para sair de um labirinto;
- 5 disposição de componentes em uma placa.

Ponto de parada: busca local

Retomando

Até aqui, mudamos a pergunta: em vez de “qual caminho leva à meta?”, passamos a perguntar “qual estado tem melhor qualidade?”.

- Hill climbing explora melhorias locais.
- Simulated annealing aceita pioras controladas para escapar de ótimos locais.
- Beam search e algoritmos genéticos mantêm múltiplos candidatos.

Para pensar

Até agora, assumimos que o agente sabe o que cada ação fará. O que muda se uma mesma ação puder gerar resultados diferentes?

Retomando as propriedades do ambiente

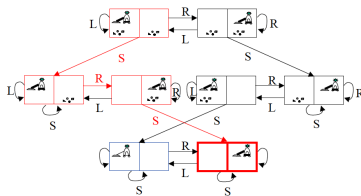
Retomando

Na aula de agentes, classificamos ambientes como observáveis ou parcialmente observáveis, determinísticos ou não determinísticos, conhecidos ou desconhecidos.

- A busca clássica estudada na Parte I pressupõe um cenário particularmente favorável.
- O agente sabe em qual estado está.
- O agente conhece as ações e seus efeitos.
- Nessas condições, uma solução pode ser apenas uma sequência fixa de ações.

Quando uma sequência fixa de ações é suficiente?

- Ambiente totalmente observável.
- Ações determinísticas.
- Modelo do ambiente conhecido.
- Estado atual conhecido pelo agente.



Observação

Se essas hipóteses deixam de valer, a solução pode precisar depender do que o agente observar durante a execução.

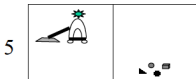
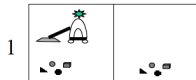
Exemplo: aspirador errático

- A ação *Suck* pode ter efeitos diferentes:
 - em uma posição suja, pode limpar apenas a posição atual ou também a adjacente;
 - em uma posição limpa, pode eventualmente introduzir sujeira.
- Assim, um estado e uma ação podem levar a um conjunto de estados possíveis.
- A solução precisa prever o que fazer depois de observar qual resultado ocorreu.

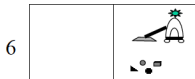
Observação

Um plano de contingência contém decisões condicionais do tipo “se ocorreu este estado, faça X; caso contrário, faça Y”.

Espaço de estados do aspirador

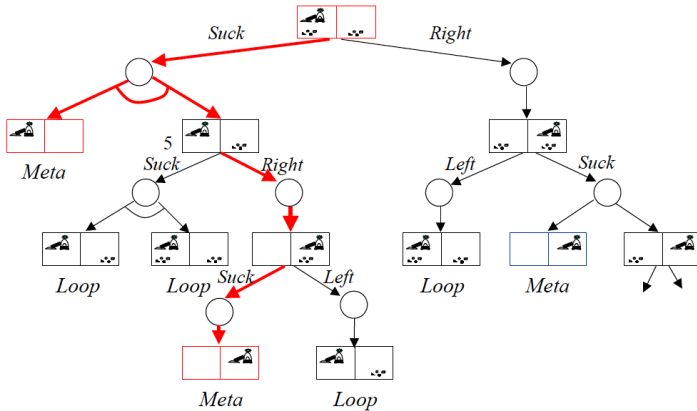


meta



meta

Árvore AND-OR



Por que AND e OR?

- Em um nó **OR**, o agente escolhe uma ação entre alternativas possíveis.
- Em um nó **AND**, a solução precisa considerar **todos** os resultados possíveis da ação escolhida.
- Uma solução em uma árvore AND-OR é uma subárvore que:
 - escolhe uma ação em cada nó OR;
 - inclui todos os descendentes necessários em cada nó AND;
 - termina em metas para todos os resultados relevantes.

Pergunta

Qual é a diferença entre “escolher uma alternativa” e “garantir uma resposta para todas as consequências”?

Algoritmo de busca AND-OR

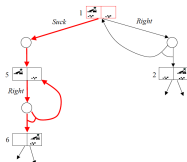
function AND_OR_GRAPH_SEARCH(*problem*) **returns** a conditional plan, or failure
 OR_SEARCH(*problem*.INITIAL-STATE, *problem*, [])

function OR_SEARCH(*state*, *problem*, *path*) **returns** a conditional plan, or failure
 if *problem*.GOAL-TEST(*state*) **then return** the empty plan
 if *state* is on *path* **then return failure**
 for each *action* **in** *problem*.ACTIONS(*state*) **do**
 plan $\leftarrow$ AND_SEARCH(RESULTS(*state*, *action*), *problem*, [*state*|*path*])
 if *plan* $\neq$ *failure* **then return** [*action*|*plan*]
 return failure

function AND_SEARCH(*states*, *problem*, *path*) **returns** a conditional plan, or failure
 for each s_i **in** *states* **do**
 *plan*_{*i*} $\leftarrow$ OR_SEARCH(s_i , *problem*, *path*)
 if *plan*_{*i*} = *failure* **then return failure**
 return [if s_1 then *plan*₁ else if s_2 then *plan*₂ else....if s_{n-1} then *plan*_{*n-1*} else *plan*_{*n*}]

Exemplo: ação de movimento pode falhar

- Considere que *Right* e *Left* possam falhar e deixar o agente na mesma posição.
- Exemplo: $RESULTS(1, Right) = \{1, 2\}$.
- Uma solução útil pode ser cíclica: repetir a ação até que ela funcione.
- Planos cíclicos exigem cuidado, pois algoritmos AND-OR simples podem tratar ciclos como falha.



[Suck, S_1 : Right, if State = 5 then S_1 else Suck]

while State = 5 do Right

Discussão: de caminho para estratégia

Discussão

Em um ambiente determinístico, podemos planejar “A, depois B, depois C”. Em um ambiente não determinístico, precisamos planejar “A; se acontecer X, faça B; se acontecer Y, faça C”.

Pergunta

O que acontece se, além de não controlar completamente os resultados, o agente também não souber exatamente em qual estado está?

Observação parcial e estado de crença

- Em um ambiente parcialmente observável, o agente pode não saber qual é o estado físico verdadeiro.
- O **estado de crença** representa o conjunto de estados físicos que ainda são compatíveis com as ações e percepções observadas.
- Em vez de buscar diretamente no espaço de estados físicos, podemos buscar no espaço de estados de crença.
- Cada ação e cada percepção alteram o conjunto de possibilidades considerado pelo agente.

Pergunta

Se o agente considera três posições possíveis, qual delas deve usar para decidir sua próxima ação?

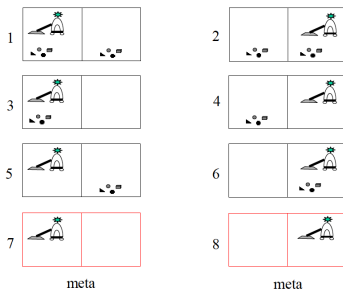
Exemplo: busca sem observações

- Considere um agente sem sensores: suas percepções não informam onde ele está ou onde há sujeira.
- O agente conhece a estrutura do mundo, mas o estado inicial pode ser qualquer um dos oito estados do aspirador.
- O estado de crença inicial é, portanto, $\{1, 2, 3, 4, 5, 6, 7, 8\}$.
- Uma sequência de ações deve funcionar independentemente de qual desses estados físicos seja o verdadeiro.

Observação

Esse tipo de problema também é chamado de busca sem observações ou problema conformante.

Reduzindo a incerteza por ações


$$\begin{aligned} \text{Right} &\rightarrow \{2, 4, 6, 8\} \\ [\text{Right}, \text{Suck}] &\rightarrow \{4, 8\} \\ [\text{Right}, \text{Suck}, \text{Left}, \text{Suck}] &\rightarrow \{7\} \end{aligned}$$

Pergunta

Como uma sequência de ações pode reduzir a incerteza mesmo sem receber novas observações?

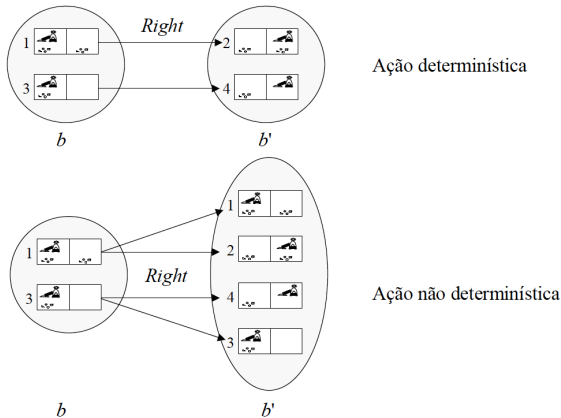
Espaço de estados de crença

- Se o problema físico possui N estados, existem até 2^N subconjuntos possíveis.
- Cada subconjunto pode representar um estado de crença.
- Nem todos os 2^N estados de crença são necessariamente atingíveis.
- A explosão combinatória pode ser ainda maior do que no espaço físico original.

Observação

Representar incerteza explicitamente aumenta o poder do agente, mas também pode aumentar muito o espaço de busca.

1000



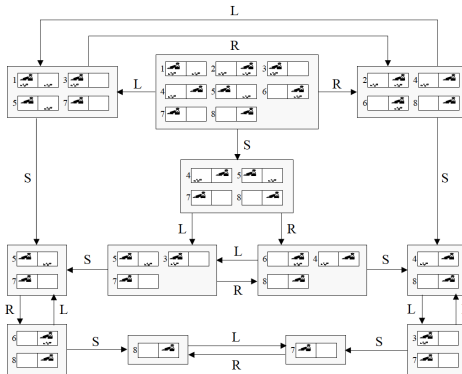
Meta em um estado de crença

- O agente procura um plano que funcione com segurança para todos os estados ainda possíveis.
- Um estado de crença b satisfaz a meta quando todos os seus estados físicos satisfazem o teste de meta.
- Assim, não basta que **algum** estado do conjunto seja uma meta.
- A solução precisa ser robusta à incerteza representada em b .

Pergunta

Se o agente acredita estar em dois estados e apenas um deles é meta, ele pode considerar o problema resolvido?

Estados de crença atingíveis



Observação

No exemplo do aspirador, apenas uma parte dos $2^8 = 256$ estados de crença possíveis é atingível pelas ações disponíveis.

Quando o agente possui sensores

- A percepção fornece informação sobre o estado físico verdadeiro.
- Uma função de percepção relaciona estados físicos às observações possíveis.
- Exemplo: observar $[A, Dirty]$ pode eliminar estados incompatíveis com estar em A e perceber sujeira.
- Depois de agir, o agente primeiro prevê estados possíveis e depois usa a percepção para eliminar hipóteses incompatíveis.

Pergunta

A percepção cria novos estados possíveis ou elimina possibilidades incompatíveis com o que foi observado?

9 7 1 1 3 9

1 Predição:

$$\hat{b} = \text{PREDICT}(b, a).$$

- 2 Observação:** o ambiente produz uma percepção o compatível com o estado real.

3 Atualização:

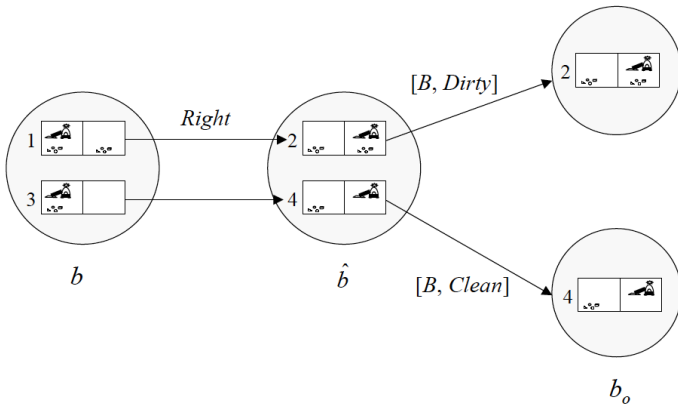
$$b' = \text{UPDATE}(\hat{b}, o).$$

$$b' = \text{UPDATE}(\text{PREDICT}(b, a), o)$$

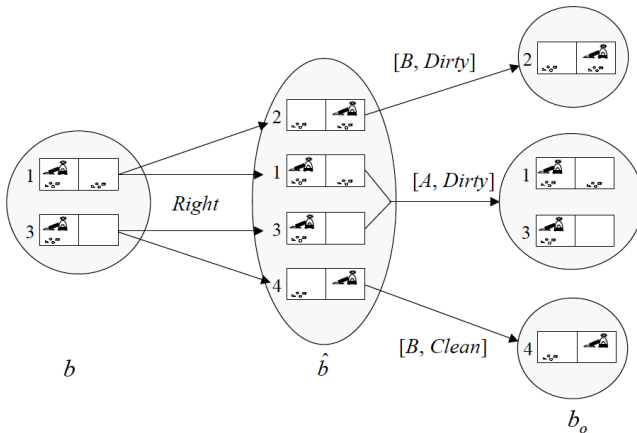
Observação

A ação pode aumentar a incerteza; a percepção pode reduzi-la.

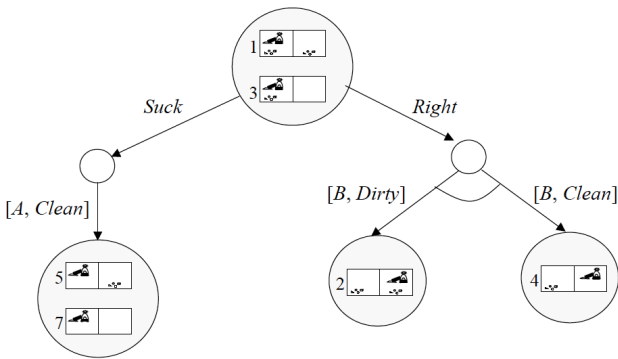
Transição com ação determinística



Transição com ação não determinística



Plano para um ambiente parcialmente observável



Observação

A solução pode combinar ações e testes condicionais baseados no estado de crença ou na percepção recebida.

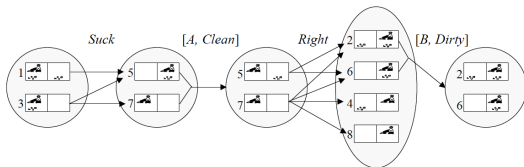
Agente em ambiente parcialmente observável

- Formula o problema e busca um plano, como no agente clássico de solução de problemas.
- Mantém internamente um estado de crença.
- Executa uma ação.
- Recebe uma percepção do ambiente.
- Atualiza sua crença:

$$b' = \text{UPDATE}(\text{PREDICT}(b, a), o).$$

- Escolhe a continuação apropriada do plano condicional.

Ciclo de predição e atualização

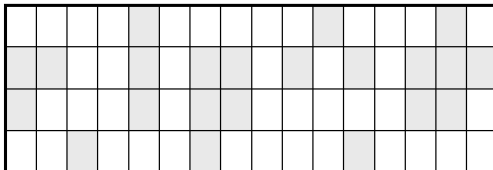


$$b' = \text{UPDATE}(\text{PREDICT}(b, a), o)$$

Pergunta

Em qual etapa o agente usa o modelo do ambiente? Em qual etapa ele usa os sensores?

Exemplo: localização



robô



obstáculos



sensores: 4 sonares

detectam obstáculos

direção obstáculo: N, S, W ou E

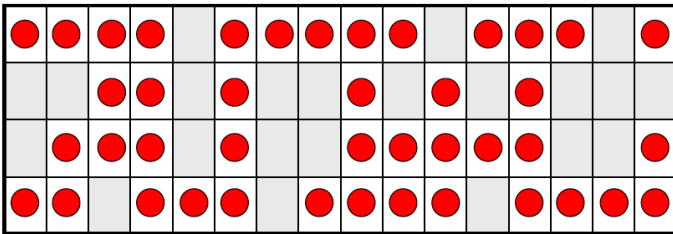
ação: *Move* (posição adjacente aleatória)

Posição atual do robô ?

Observação

Localização é um exemplo direto de manutenção de estado de crença: o robô pode começar sem saber sua posição exata.

Localização: crença inicial

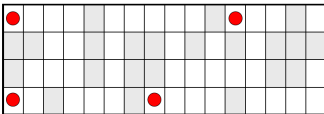
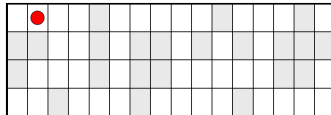


estado inicial: b

Pergunta

O conjunto de pontos vermelhos representa posições simultâneas do robô ou hipóteses alternativas sobre uma única posição real?

Localização: atualização com percepção

$$o = NSW$$

$$b_o = \text{UPDATE}(b, o)$$


Observação

A cada ciclo de movimento e observação, hipóteses incompatíveis podem ser removidas do estado de crença.

Busca online

- Em busca offline, o agente procura uma solução antes de executar as ações relevantes no ambiente.
- Em busca online, planejamento e execução são intercalados:
ação → observação → decisão → ação → ...
- É necessária quando o ambiente é inicialmente desconhecido ou quando os efeitos das ações precisam ser descobertos durante a execução.
- As ações também funcionam como experimentos para adquirir conhecimento.

Pergunta

Como escolher uma ação quando sua consequência ainda não é conhecida pelo agente?

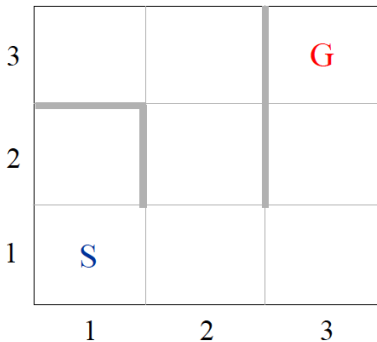
O que um agente online pode conhecer?

- $ACTIONS(s)$: ações disponíveis no estado atual.
- $GOAL-TEST(s)$: se o estado atual satisfaz a meta.
- $h(s)$: uma estimativa de distância ou custo até a meta, quando disponível.
- $RESULT(s, a)$: pode ser desconhecido até que a ação seja executada.
- $c(s, a, s')$: o custo pode ser conhecido ou observado depois da transição.

Observação

O agente precisa simultaneamente alcançar a meta e aprender a estrutura relevante do ambiente.

Exemplo de busca online

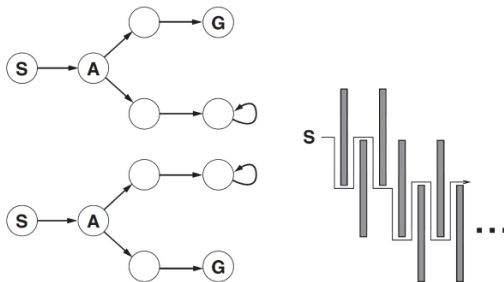


S estado inicial

G meta

- O agente conhece ações como subir, descer, esquerda e direita.
- Pode usar distância de Manhattan como heurística.
- Mas só descobre algumas transições ao executar as ações.

Custo de exploração e razão competitiva



- Um agente online pode precisar explorar caminhos que um agente com mapa completo evitaria.
- A razão competitiva compara o custo do agente online com o custo de uma solução obtida com conhecimento completo do ambiente.

Busca em profundidade online

```

function ON_LINE_DFS_AGENT ( $s'$ ) returns an action
inputs:  $s'$ , a percept that identifies the current state
persistent: result, table indexed by state and action, initially empty
               untried, table that lists, for each state, actions not yet tried
               unbacktracked, table that lists, for each state, backtracks not yet tried

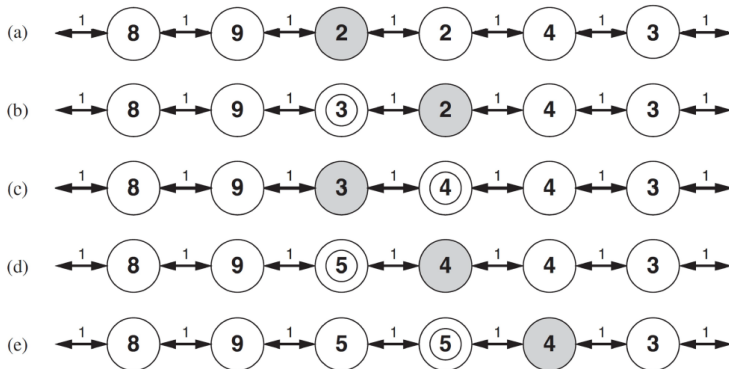
if GOAL-TEST( $s'$ ) then return stop
if  $s'$  is a new state (not in untried) then untried [ $s'$ ]  $\leftarrow$  ACTIONS( $s'$ )
if  $s$  is not null then
    result [ $s, a$ ]  $\leftarrow s'$ 
    add  $s$  to the front of unbacktracked [ $s'$ ]
if untried [ $s'$ ] is empty then
    if unbacktracked [ $s'$ ] is empty then return stop
    else  $a \leftarrow$  an action  $b$  such that result [ $s', b$ ] = POP(unbacktracked [ $s'$ ])
else  $a \leftarrow$  POP(untried [ $s'$ ])
 $s \leftarrow s'$ 
return  $a$ 

```

Observação

A busca online precisa registrar experiências: estados visitados, ações ainda não testadas e transições observadas.

Busca online local e aprendizagem



Observação

Uma estratégia online pode atualizar estimativas heurísticas a partir da experiência obtida durante a exploração.

Atualizando a heurística durante a busca

function LRTA*_AGENT (s') **returns** an action

inputs: s' , a percept that identifies the current state

static: *result*, table indexed by state and action, initially empty

H, a table of costs estimates indexed by state, initially empty

s, *a*, the previous state and action, initially null

if GOAL-TEST (s') **then return** stop

if s' is a new state (not in *H*) **then** $H[s'] \leftarrow h(s')$

if *s* is not null

result [*s*,*a*] $\leftarrow s'$

$H[s] \leftarrow \min_{b \in \text{ACTIONS}(s)} \text{LRTA}^*_{\text{COST}}(s, b, \text{result}[s, b], H)$

$b \in \text{ACTIONS}(s)$

$a \leftarrow$ action *b* in $\text{ACTIONS}(s')$ that minimizes $\text{LRTA}^*_{\text{COST}}(s', b, \text{result}[s', b], H)$

$s \leftarrow s'$

return *a*

function LRTA*_COST (*s*, *a*, s' , *H*) **returns** a cost estimate

if s' is undefined **then return** $h(s)$

else return $c(s, a, s') + H[s']$

Pergunta

Qual é a diferença entre usar uma heurística definida antes da busca e aprender uma estimativa enquanto o agente interage com o ambiente?

Atividade final: qual modelo de busca usar?

Atividade

Para cada cenário, identifique as propriedades relevantes do ambiente e proponha uma abordagem:

- 1 GPS com mapa conhecido e tempos de viagem estimados;
- 2 escalonamento de tarefas sem interesse na sequência de soluções intermediárias;
- 3 robô cujo comando de movimento pode falhar;
- 4 robô que recebe apenas sensores locais e não conhece sua posição exata;
- 5 agente que entra em um labirinto sem possuir o mapa.

Pergunta

Em qual cenário a solução é um caminho, um estado, um plano condicional, um estado de crença ou uma política de exploração?

Síntese da Aula 4

- A organização da fronteira define as estratégias clássicas de busca.
- Heurísticas incorporam conhecimento do domínio e podem reduzir a exploração.
- Busca local é adequada quando o estado final importa mais que o caminho.
- Não determinismo exige considerar diferentes resultados de uma mesma ação.
- Observação parcial exige manter estados de crença.
- Ambientes desconhecidos exigem intercalar busca, ação, observação e aprendizagem.

Pergunta

Qual hipótese sobre o ambiente, quando removida, provoca a maior mudança na forma como um agente deve buscar uma solução?

Dúvidas



Dúvida é o começo da sabedoria.